Testing Terraform Infrastructure-as-code: Unit tests & BDD end-to-end scenario testing review

Jaroslav Pantsjoha

Welcome back! This is another technical post, building up on the previous **terraform** and **kubernetes** Infrastructure-as-code themed write-ups, we have at Contino.

> TL;DR
> Whatever your team size. Implementation of a good terraform-based infrastructure configuration analysis, and end-to-end sanity testing, does not need to be long nor complicated.

I was fortunate enough to receive a great challenge to investigate, develop and deliver a suitable open source testing framework for the terraform code base, as part of the infrastructure release pipeline. The “Quality Assurance for everything” intent is nothing new. The implementation however, does depend and varies across organisational infrastructure maturity and risk tolerances, it may be comfortable with, — before [any such] go-live stages.

Taking a fresh look at this are of “testing the code for infrastructure” again, — showcased all the new, all the shiny test frameworks, — and a much welcomed opportunity for self-update.

I must admit, I did have a degree of bias when getting started, thinking that perhaps, this “enterprise-grade assurance” may cost a lot of effort to setup and provision overhead in itself.

The Hashicorp Terraform provides sufficient functionality out of the box, to check and validate your codebase after-all;

- Lint — `terraform fmt -check` and `terraform validate`
- Preview — `terraform plan`
- Build — `TF_LOG=debug terraform apply` for all the nitty

gritty details.

Terraform Static Code Analysis Tools

A fresh Google search on the topic of relevant terraform test tooling revealed as much, and the list of tools available is quite vast.

But we do have a particular **requirements list**,

- **A** need for the terraform resource configuration to have **Unit Tests**, and an option to extend any such generic list of best practices* checks, of a given public cloud provider. Lastly, ease of use is what we're after to get started immediately.

> ** no ec2 instances open to the world 0.0.0.0/0, — and the like.*

- **T**he ability to run a more bespoke tests, — a form of **compliance "guard rails" to enable me to constrain resource configuration**. This option needs to be reasonably extensible to safeguard subnets, security roles, bespoke tagging, and even ensure certain EKS cluster feature flags are enabled or otherwise.

- **A** more personal experience driven requirement, — need to ensure the test framework attempts to reduce the engineering overhead where possible, not increases it, skill silos included. I am quite mindful that such test suites — once created, may become the engineering overhead, even a debt, to upkeep and extend later.
- **C**ommunity support is important, as far as Open Source Software option is concerned. At the very least either frameworks chosen should be on the common development language, like Go or Python. If need arises for framework deep dive and custom method extensions, *it's perhaps more likely to find a platform engineer with one rather than two development language experience*. All and any previous experience, the current team and the wider organisation may have

had, with similar tools is also helpful. And so, *I was set and ready hunting for the right test tool & frameworks for the job*.

lets get hunting

Shortlist, Terraform test frameworks and tools to review and compare

Hope this list below helps you on your own IaC static analysis and QA journey. Note this is a full list of all terraform testing relevant tools discovered, which is a mix bag of configuration sanity **testing** per se, **lint** tools, and secOps-oriented best-practices, with **unit-tests**. The list is for your reference.

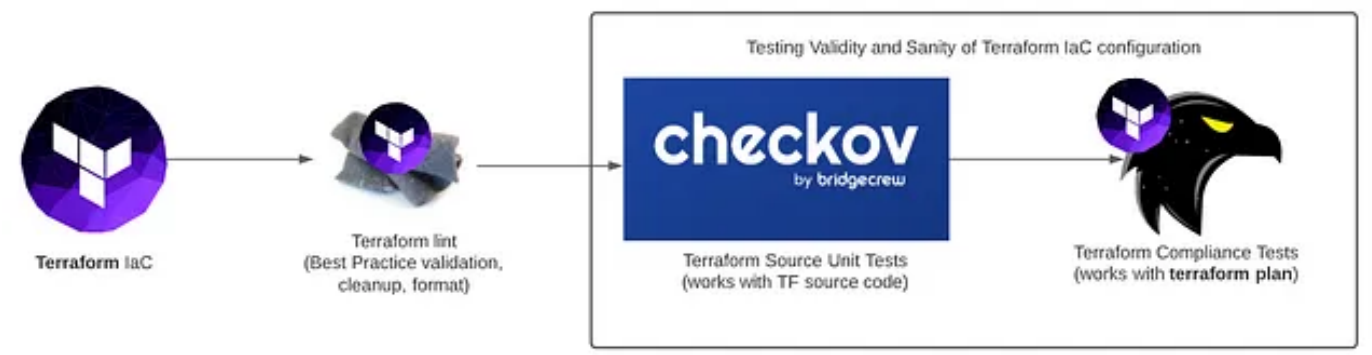
- **tflint** — https://github.com/terraform-linters/
- **terrafirma** — https://github.com/wayfair/terrafirma
- **tfsec** — https://github.com/liamg/tfsec
- **terrascan** — https://github.com/cesar-rodriguez/terrascan (no TF 0.13 support at this time)
- **checkov** — https://github.com/bridgecrewio/checkov/
- **conftest** — https://github.com/instrumenta/conftest

The Chosen Test Tools

Now to recap, I was keen to settle on the **standardised unit test** for the terraform resource components and, a more customisable suite of tests, which take the resource configuration to validate on, based on the `terraform plan` outcome.

After reviewing a number of pros and cons of each, I settled on `checkov` and aptly named `terraform-compliance` which are both python based frameworks. This has appeared to deliver all my aforementioned requirements.

The IaC release pipeline, in a nutshell, looks something like this.



With the deep-dive into the frameworks, I have inevitably self-examined my own experience and the current findings to date on this matter, such as;

- Sanity tests need not cost the world
- The maturity of some test frameworks offer a great deal of **best practice** as far as unit tests are concerned, out of the box.
- Getting started with most are really straight forward, and thus acceptance and integration of such test frameworks are a must for an organisation of any size, where IaC compliments the organisation's very own Agile, backed by "fail fast" and " go faster" business requirements.

Unit Testing — Checkov by BridgeCrew

https://www.checkov.io/

> ***Checkov is a static code analysis tool for infrastructure-as-code.***
>
> *It scans cloud infrastructure provisioned using **Terraform**, Cloudformation, **Kubernetes**, Serverless or ARM Templates and detects security and compliance misconfigurations.*

There are a number of default best-practice unit tests when scanning your terraform code repository will highlight deviation from best practices — such as having VM a port 22 open to the world (0.0.0.0/0) for example, evident from the security configuration.

All tests are available here on the handy GitHub link.

Getting started is really straight forward.

- Install the binary
- Initialise terraform directory `terraform init`
- Run **chechov** on that directory

All the default unit tests available can be listed with the handy CLI liner. Alternatively, when **checkov** runs, it will output all passing and failing unit tests by default. Very handy, simple to get started. Best practices of `terraform` validated, but not everything. This is the fundamental

difference.

Chechov will happily assess your terraform code ONLY. It can run right after `terraform init`. It does not care about your `terraform plan` — potential pros and cons here, and it does what it says on the tin — “Static code analysis”. Be mindful of the implications, and any logic consideration for your resources.

checkov -l # most helpful liner to view ALL as-is shipped checks available

output on checkov run, highlighting what checks it passes or fails, as applicable.

Additional unit tests can be written, mind, as long as you are comfortable to deep-dive with Python development. The framework’s development language was one of the requirements, given I did have to explore the test codebase at times, to assess how much effort [any] such additional methods would cost. That and the maintenance

consideration for the wider team, versus alternative framework achieving the same purpose, is one of the drivers in the decision making process, here.

To recap, as far as static-code-analysis is concerned, `checkov` is round great. Considering the case where I want certain IP address of subnet whitelisted from get-go, this is not a place for e2e tests however, and require a separate test framework.

Though sure, I can replicate the unit test and tune the values, calling it a day, baking in the hard values for my subnet/IP. But what if, — I have several instances and projects — do I start skipping the test where it is relevant? Perhaps. Perhaps not.

> This is where the secondary test framework, terraform-compliance comes into play.

Terraform-compliance

https://terraform-compliance.com/

> *Terraform-compliance is a lightweight, security and compliance focused test framework against terraform to enable negative testing capability for your infrastructure-as-code.*

A backstory

Once again, BDD as test framework came into focuses recently highlighting the need for versatile test framework, but also something else. **Simplicity**.

In fact, I feel that this **Behaviour Driven Development** does not get enough attention. You may have heard of **TDD — Test Driven Development —** and that goes a long way back, primarily in the software development environment. But this is where the likes of BDD frameworks facilitate an additional logic to enable a simpler, concise and repeatable way to develop end-to-end customisable tests without diving deep into any new particular development language, for the typical infrastructure engineer.

While one can go out and codify pretty much anything under the sun, **it comes down to manageability**; understanding of the complexity which may require further documentation, and it goes without saying — support and maintenance, by your colleagues, going forward. Read more on BDD here.

Cucumber.io — more of a lang, facilitating this very test journey— with full intent to simplify this process realising **WYSIWYG** approach to **test creation**, **understanding** and **maintenance**. These examples are defined before the development starts, and are used as acceptance criteria.

They are part of the definition of done.

Tests with Terraform-Compliance

Each framework is considered on it's own merits and deep dive with each highlighted and caveats and nuances where they stand best complimentary — going forward, we can make use of both.

Here is the example of such Test Case developed with `terraform-compliance` framework, leveraging the **B**ehaviour **D**riven **D**evelopment. This enables testing of a reasonably complex end-to-end test cases.

The `terraform-compliance` framework leverages the `terraform plan` **output**. As a result this enables to generate the full "plans" for your releases and thoroughly test these accordingly. Whether it is the use of the correct encryption key-pair [for your cloud provider] account spoke, environment or otherwise, — a lot of creative freedoms to work with, and the most importantly — it's a **simple** act **to do** so.

Just view the steps and examples below.

- **Step 1**. Initialise the terraform directory
  # `terraform init`
- **Step 2**. Quick option, generate a `terraform plan` with

`#terraform plan -out=plan.out`

- **Step 3**. Write some tests. Ok, to be fair — there are already an examples folder. Let's go through my own test examples below, written based on my `terraform plan` output.

This outlines the snippet of the terraform plan — a terraform configuration, which creates the EKS with a specified launch group.
Let's ensure that our particular dev environment terraform IaC does not use any `instance_type` **but** the "approved" `a1.xlarge` or `a1.2xlarge`.

Now, I will change it to `t2.small` on purpose to simulate a failure.

Writing the test to ensure we can successfully validate that requirement.

- **Step 4**. Lets get `terraform-compliance` to assess the plan leveraging your test cases
  `#terraform-compliance -p plan.out -f ./<test-cases-folder>`

Run Tests

A sample result on PASS and FAIL

If our Terraform IaC features the correct `instance_type` this results in all-green **SUCCESS**

If our Terraform IaC features violates the compliance requirement featuring the incorrect `instance_type` this results in all-red **FAIL**

Let's write even more tests

Another simple tests, borrowed from the examples directory to get started

For another fail scenario, this is to highlight that on FAIL, the user gets the Actual_Value extracted and shown for reference and debug.

Test Run Results

Once all tests run in succession, there is a handy “total summary” for all test passes and fails, and whether they were skipped. I like this because this enabled me to write a long list of thorough tests and find the output clear output of what failed and when, — at the end. Additionally, some tests can be skipped-on-fail with the `@warning`tag, as per example below.

Summary

This has certainly been a great refresh of some great quality validation and test frameworks available for the Terraform Infrastructure-as-code build-out.

JP Says “Do it”

I enjoyed exploring both and particularly impressed with the simplicity of **checkov** integration, as well as the amazing e2e `terraform plan` validation and the custom testing options with `terraform-compliance.`

The latter reminds me of another great `kubernetes` BDD e2e test framework `behave` which I have worked with in the past.

An all-python test frameworks simplify a cross-framework

general python knowledge sharing, and reducing mental fatigue in [any] further maintenance and test case development in the future.

Whether you have a need for a best practice configuration check — no terraform plan required — `checkov` could be what you are after, otherwise for a more feature-rich terraform plan validation purpose — `terraform-compliance` could be that answer. Best bit, being a BDD framework, `terraform-compliance` is very straight forward to get started and get going.

Unit Test first. simplest. Checkov by bridgecrewio has ‘best practice’ validations out of the

box

Now, there really is no good reason to skip any of those Quality Assurance exercises, whatever your team size. Especially, given this small level of implementation effort required, as exemplified in the post :)
Good luck, though you really won't need it.

Right, This is it. Hope you enjoyed this. Enjoying the read?

Like, Clap and Share this post along!

PS There are quite a number of fantastic projects taking place at **Contino**. If you are looking to work on the latest-greatest infrastructure stack or looking for a challenge, — Get in touch! We're hiring, looking for bright minds at every level. At **Contino**, we pride ourselves on delivering the best practices cloud transformation projects, for medium-sized businesses to large enterprises.